

Introduction to Computing (CS 101)

Recursive Functions and Scope

T. Venkatesh & John Jose



**Department of Computer Science & Engineering
Indian Institute of Technology Guwahati**

How to write function in modular way?

Declarations

```
#include <stdio.h>
```

```
double GetTemp();  
double CelsToFahr(double);  
void DispRes(double, double);
```

```
int main(){  
    double TempC, TempF;
```

```
    TempC=GetTemp();  
    TempF=CelsToFahr(TempC);  
    DispRes(TempC, TempF);
```

```
    return 0;  
}
```

Function Calls

```
double CelsToFahr(double Tem){  
    return (Tem * 1.8 + 32.0);  
}
```

```
double GetTemp (){  
    double Temp;  
    printf("Please enter temp in  
           degrees Celsius:");  
    scanf("%lf", &Temp);  
    return Temp;  
}
```

```
void DispRes(double CTemp,  
             double FTemp){  
    printf("Original: %5.2f  
           C\n", CTemp);  
    printf("Equivalent: %5.2f  
           F\n", FTemp);  
}
```

Parameter Passing

- **Actual parameters** are the parameters that appear in the function call

```
ave =AverageTwo(value1,value2) ;
```

- **Formal parameters** are the parameters that appear in the function header

```
float AverageTwo(int num1,int num2)
```

- Corresponding actual and formal parameters must be of the same data type and are matched by position.
- Each formal parameter receives the value of its corresponding actual parameter.

Recursions: Recursive functions

- Expressing an entity in terms of itself is called recursion
- Recursive Functions – They call themselves
- This method of problem solving substitutes a given problem with another problem of the same form but simpler than the original.
- Direct recursion: function1 calls function1 itself
- Indirect recursion; function1 calls function2 which in turn calls function1. Can be extended to any number of functions.
- Two conditions must be satisfied:
 - (1) Each time a function calls itself it must be closer to a solution
 - (2) There must be a decision criterion for stopping the process or computation

Recursion Example: factorials

- $5! = 5 * 4 * 3 * 2 * 1$
- But
 - $5! = 5 * 4!$
 - $4! = 4 * 3! \dots$
- Can compute factorials recursively
- Solve base case ($1! = 0! = 1$) then plug in
 - $2! = 2 * 1! = 2 * 1 = 2;$
 - $3! = 3 * 2! = 3 * 2 * 1 = 6;$
- $N! = N * (N-1)!$
 - $7! = 7 * 6!$
 - $7! = 7 * (6 * 5 * 4 * 3 * 2 * 1 * 1)$

Recursion vs Iteration

- **Repetition**
 - Iteration: explicit loop
 - Recursion: repeated function calls
- **Termination**
 - Iteration: loop condition fails
 - Recursion: base case recognized
- **Both can have infinite loops**
- **Balance**
 - Choice between performance (iteration) and good software engineering (recursion)

Recursion Example: Factorials

- From Definition: but no termination

```
int Fact (int N) {  
  
    return N * Fact (N-1);  
}
```

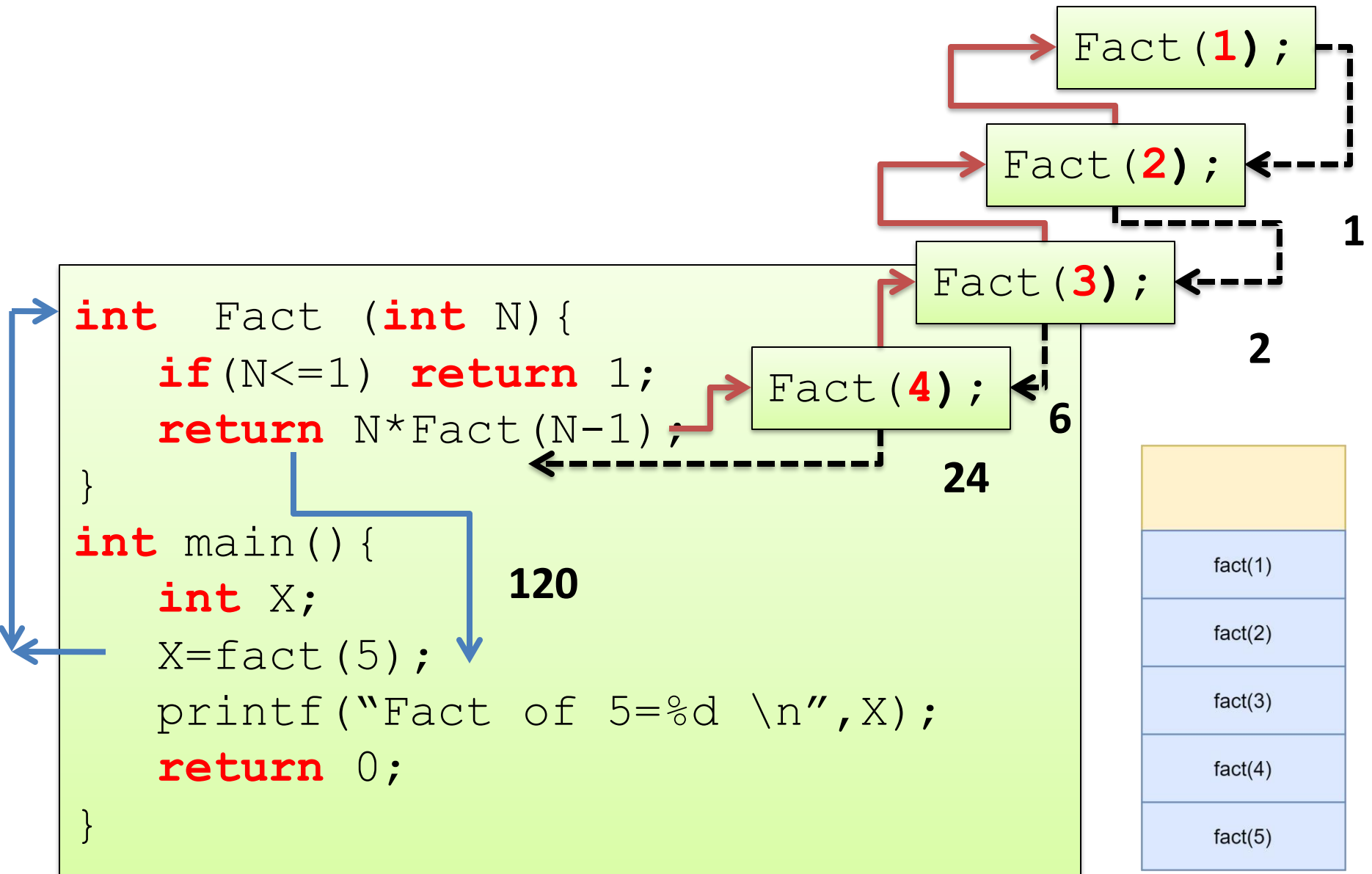


- With proper base case (termination guaranteed)

```
int Fact (int N) {  
    if (N<=1)  
        return 1;  
    return N*Fact (N-1);  
}
```



Factorial: Recursive call



Recursive call

- Nested Function call uses a stack to store the return address, return result
- Recursion
 - Stack grows as deepen the nested function call
 - Recursive call: Stack grows when it call next recursive function
 - All the local variable need to be put into stack
 - Stack grows like this: Fact Example
 - Main, Fact (5), Fact (4), Fact (3), Fact (2), Fact (1)



Factorial Function

```
int Fact (int N) {  
    if (N<=1) return 1;  
    return N*Fact (N-1) ;  
}
```

```
if (1 == 1)  
    return 1;  
else  
    return 1 * Fact(0) ;
```

```
graph TD; Fact3[Fact(3)] -- red arrow --> Fact2[Fact(2)]; Fact2 -- red arrow --> Fact1[Fact(1)]; Fact1 -- red arrow --> Fact0[Fact(0)]; Fact0 -- grey arrow --> Fact1; Fact1 -- grey arrow --> Fact2; Fact2 -- grey arrow --> Fact3;
```

```
if (2 == 1)  
    return 1;  
else  
    return 2 * Fact(1) ;
```

```
if (3 == 1)  
    return 1;  
else  
    return 3 * Fact(2) ;
```

```
main {  
    X = Fact(3)  
}
```

Activation Stack for Factorial

Call the function: `X = Fact(5);`

Main Function: Unfinished: <code>X = Fact(5);</code>

Activation Stack for Factorial

Fact. 1st: N=5,	Unfinished: $5 * \text{Fact}(4)$
-----------------	----------------------------------

Main Function: Unfinished: $X = \text{Fact}(5);$
--

Activation Stack for Factorial

Fact. 2nd: N=4,	Unfinished: $4 * \text{Fact}(3)$
-----------------	----------------------------------

Fact. 1st: N=5,	Unfinished: $5 * \text{Fact}(4)$
-----------------	----------------------------------

Main Function:	Unfinished: $X = \text{Fact}(5);$
----------------	-----------------------------------

Activation Stack for Factorial

Fact. 3rd: N=3,	Unfinished: $3 * \text{Fact}(2)$
-----------------	----------------------------------

Fact. 2nd: N=4,	Unfinished: $4 * \text{Fact}(3)$
-----------------	----------------------------------

Fact. 1st: N=5,	Unfinished: $5 * \text{Fact}(4)$
-----------------	----------------------------------

Main Function:	Unfinished: $X = \text{Fact}(5);$
----------------	-----------------------------------

Activation Stack for Factorial

Fact. 4th: N=2,	Unfinished: $2 * \text{Fact}(1)$
-----------------	----------------------------------

Fact. 3rd: N=3,	Unfinished: $3 * \text{Fact}(2)$
-----------------	----------------------------------

Fact. 2nd: N=4,	Unfinished: $4 * \text{Fact}(3)$
-----------------	----------------------------------

Fact. 1st: N=5,	Unfinished: $5 * \text{Fact}(4)$
-----------------	----------------------------------

Main Function:	Unfinished: $X = \text{Fact}(5);$
----------------	-----------------------------------

Activation Stack for Factorial

Fact. 5th: N=1,	Finished: returns 1
Fact. 4th: N=2,	Unfinished: $2 * \text{Fact}(1)$
Fact. 3rd: N=3,	Unfinished: $3 * \text{Fact}(2)$
Fact. 2nd: N=4,	Unfinished: $4 * \text{Fact}(3)$
Fact. 1st: N=5,	Unfinished: $5 * \text{Fact}(4)$
Main Function:	Unfinished: $X = \text{Fact}(5);$

Activation Stack for Factorial

Fact. 4th: N=2,	Finished: returns $2*1$
Fact. 3rd: N=3,	Unfinished: $3*Fact(2)$
Fact. 2nd: N=4,	Unfinished: $4*Fact(3)$
Fact. 1st: N=5,	Unfinished: $5*Fact(4)$
Main Function:	Unfinished: $X = Fact(5);$

Activation Stack for Factorial

Fact. 3rd: N=3,	Finished: returns $3*2$
-----------------	-------------------------

Fact. 2nd: N=4,	Unfinished: $4*Fact(3)$
-----------------	-------------------------

Fact. 1st: N=5,	Unfinished: $5*Fact(4)$
-----------------	-------------------------

Main Function:	Unfinished: $X = Fact(5);$
----------------	----------------------------

Activation Stack for Factorial

Fact. 2nd: N=4,	Finished: returns $4*6$
-----------------	-------------------------

Fact. 1st: N=5,	Unfinished: $5*Fact(4)$
-----------------	-------------------------

Main Function:	Unfinished: $X = Fact(5);$
----------------	----------------------------

Activation Stack for Factorial

Fact. 1st: N=5,	Finished: returns 5*24
-----------------	------------------------

Main Function: Unfinished: X = Fact (5);

Activation Stack for Factorial

Main Function: finished: $X = 120$;

Sum of N Natural Numbers

```
#include <stdio.h>
int sum(int n);
void main() {
    int num, total;
    printf("Enter a positive number: ");
    scanf("%d", &num);
    total = sum(num);
    printf("Sum of %d natural nos is:%d\n",
        num, total);
}
int sum(int n) {
    if ( n == 0)
        return 0;
    else
        return n + sum(n-1);
}
```

Fibonacci Number Sequence

if $n = 1$, then $\text{Fib}(n) = 1$

if $n = 2$, then $\text{Fib}(n) = 1$

if $n > 2$, then $\text{Fib}(n) = \text{Fib}(n-2) + \text{Fib}(n-1)$

Numbers in the series:

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

```
int  Fib  (int  n) {  
    if  (n == 1)  || (n == 2) )  
        return 1;  
    else  
        return Fib (n-2)+Fib (n-1) ;  
}  
main() {  
    int  answer;  
    answer=Fib(5) ;  
    printf("5 th Fib Num is %d",  answer);  
}
```

Operators

Arithmetic operators		Relational operators	
+	Addition	<	Is less than
-	Subtraction	>	Is greater than
*	Multiplication	<=	Is less than or equal to
/	Division	>=	Is greater than or equal to
%	Modulo division	==	Is equal to
Bitwise operators		!=	Is not equal to
&	Bitwise AND	Logical operators	
	Bitwise OR	&&	Logical AND
^	Bitwise exclusive OR		Logical OR
<<	Shift left	!	Logical NOT
>>	Shift right		
~	One's complement		

Bitwise operators

a 0000 0000 0000 1101

b 0000 0000 0000 0111

a&b 0000 0000 0000 0101 : AND

a|b 0000 0000 0000 1111 : OR

a^b 0000 0000 0000 1010 : XOR

a 0000 0000 0000 1101

a<<3 0000 0000 0110 1000 : Left shift – insert 0s

Shift left by 1 has effect of multiplying by 2

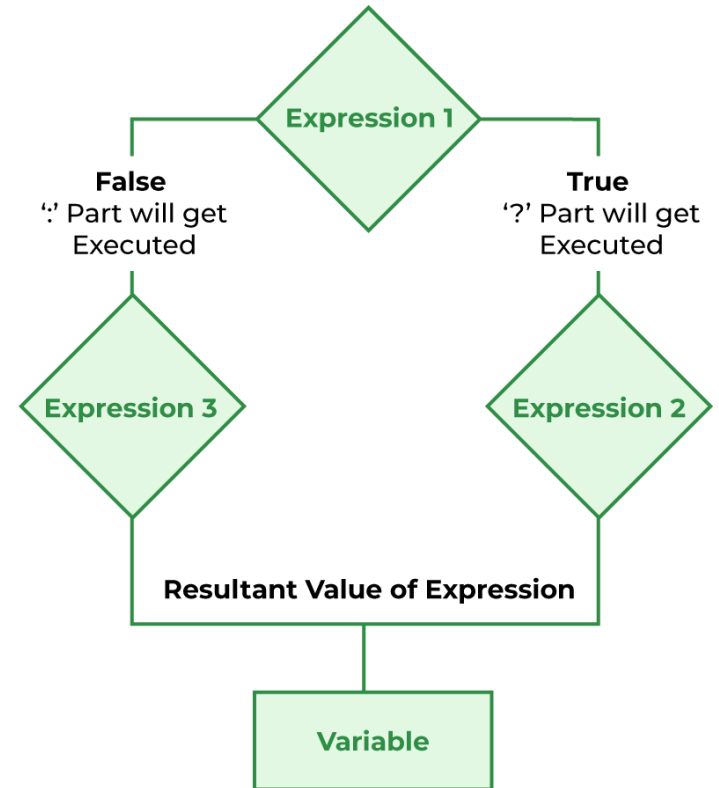
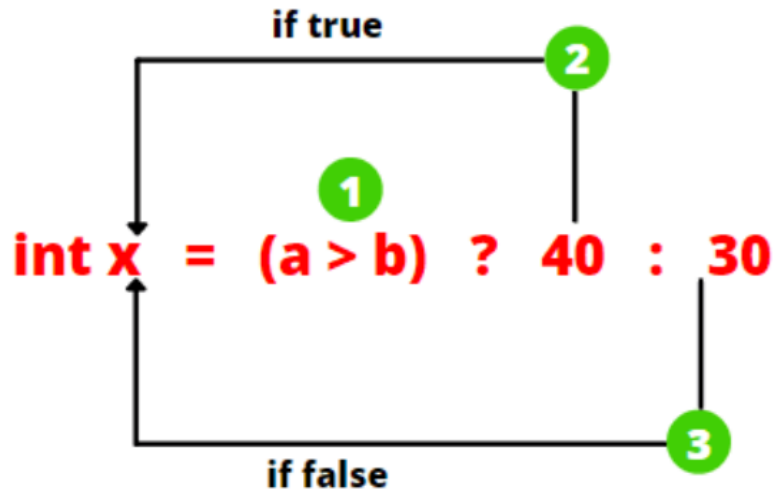
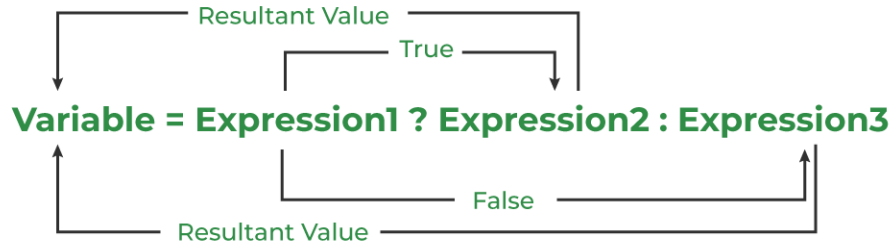
a>>2 0000 0000 0000 0011 : Right shift – drop-off

Shift right by 1 has effect of dividing by 2

Bitwise operators

- $a=3$ 0000 0000 0000 0011
- $a=-3$ 1111 1111 1111 1101
- $a>>2$ 1111 1111 1111 1111 : insert 1s: i.e. Sign
- For unsigned int, in shift right insert 0s
- Bitwise complement ($\sim$)
- Example: Clear 4-LSB bits of i
- $i = i \& 0\text{fff}0$; : assumes int=16 bits
- $i = (i >> 4) << 4$; : involves 2 shifts
- $i = i \& \sim 0\text{xf}$; : Efficient method. 0Xf has 4-LSB bits=1 and its complement has them zero

Conditional Operator (?:)



Conditional Operator (?:)

- $x = (b > 8) ? b << 3 : (b > 4) ? b >> 1 : b$

if (b>8)

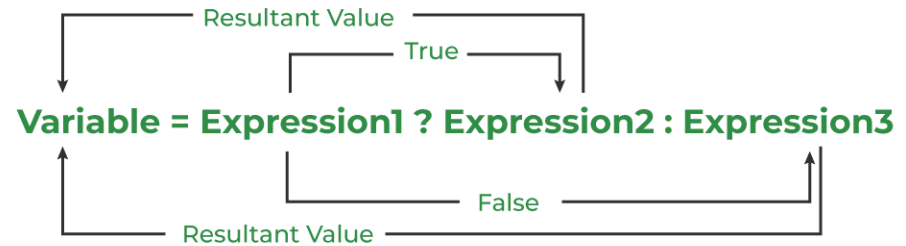
 x=b<<3;

else if (b>4)

 x=b>>1

else

 x=b



Scope and Extent

- The **region of source code** over which the declaration of an identifier is **visible is called the scope** of the identifier.
- The **period of time** during which memory is associated with a variable is **called the extent** of the variable.
- Variables declared **outside** all functions are called **global variables**.

Example: Scope

```
#include <stdio.h>
```

```
void func() {  
    int j = 20;  
    printf("j is a variable is func.\n");  
    printf("Value of j is %d\n", j);  
}
```

i and j are local variables,
Their visibility is restricted to the
function in which they are declared.

```
void main() {  
    int i = 10;  
    printf("i is a variable in main.\n");  
    printf("Value of i is %d\n", i);  
    printf("Calling func...\n");  
    func();  
    /* this stmt gives compilation error */  
    j=30;  
}
```

Example: Global variable

```
#include <stdio.h>
// g is a global variable
int g;
void func()
{
    printf("In func, g is
        visible \n");
    printf("Value of g in
        func is %d\n", g);
    g++;
}
```

```
void main() {
    printf("In main, g is
        visible \n");
    g = 20;
    printf("Value of g in
        main is %d\n", g);
    func();
    printf("In main
        again, value of g is
        %d\n", g);
}
```

Scope in nested blocks

```
#include <stdio.h>
void main() {
    int i =144, j=132;
    printf("i = %d\n", i);
    {
        /* nested block */
        int k = 12;
        printf("Enter a value for k: ");
        scanf("%d", &k);
        i = i%k;
    }
    if( i==0 )
        printf("i is a divisor of %d\n", k);
}
```

K unknown here. No scope

Scope in nested block

```
#include <stdio.h>
void main() {
    int iNum=5, jNum=30;
    printf("Before the block, iNum=%d,
jNum=%d\n", iNum, jNum);
    {
        int iNum = 10;
        printf("Inside the block, iNum=%d,
jNum=%d\n", iNum, jNum);
    }

    printf("Outside the block, iNum=%d,
jNum=%d\n", iNum, jNum);
}
```

Before the block, iNum=5, jNum=30
Inside the block, iNum=10, jNum=30
Outside the block, iNum=5, jNum=30

Scope Rule Example

```
int A; //global
int main() {
    A=1;
    MyProc();
    printf("A=%d\n", A);
    return 0 ;
}
```

```
void myProc() {
    int A=2;
    while(A==2) {
        int A=3;
        printf("A=%d\n", A);
        break;
    }
    printf("A=%d\n", A);
}
```

Outer blocks "hidden" from inner blocks if there is a variable with the same name in the inner block

Printout:

A = 3

A = 2

A = 1

Global and Static Variables

- Variables **declared outside** all functions are called global variables
- Global variables are of two types:
 - **Static** : scope limited to the file in which it is declared, Keep value after function ends
 - **Global**: which can be accessed across functions which are linked together

```
#include <stdio.h>
```

```
int iGlobal = 35, Number = 47;
```

```
void main() {
```

```
    int Number = 1000;
```

```
    printf("In Main Number is %d\n", Number);
```

```
    printf("global iGlobal is %d\n", iGlobal);
```

```
}
```

```
In Main Number is 1000  
global iGlobal is 35
```

Global and Static Variables

```
#include <stdio.h>

void printCount() {
    static int count = 1;
    /* count is initialised only once */
    printf("Count = %d\n", count);
    count = count + 1;
    /* the incremented value of count retained */
}

void main() { /* count is not accessible in main
function */
    printCount();
    printCount();
    printCount();
}
```

Count = 1
Count = 2
Count = 3

Scope and Life : Static Vs Global

```
int GA; //global
int main() {
    int i;
    GA=1;
    for (i=1; i<10; i++)
        MyProc();
    printf("GA=%d", GA);
    return 0;
}

void myProc() {
    static int SA=2;
    SA=SA+1;
}
```

**Both SA and GA
Variables exist for entire
program execution**

**SA initialized once
SA can be accessible
from myProc only
But GA accessible from
any part of Program**

Extent

- The period of time during which memory is associated with a variable is called the extent of the variable
- Storage classes define the extent of a variable:
 - **Auto**: default. Can be accessed within the function or nested block in which it is declared
 - **Register**: prefix register. Are stored in registers of the microprocessor. Usage: `register int index;` This will keep loop index variable which is accessed more often, in the register. Helps to improve performance.
 - **Static**: - Declared **within a function** are static to the function
 - **Extern**: global variable is accessible in all functions inside the file. If you want this variable to be **accessible across different files**, use the keyword **extern**. For extern variables, only one file must have the variable defined.

Thanks